

Zero-Trust Semantic Code Auditing with TypeSafe Jev

An Independent Typed Governance Architecture
for Autonomous Coding Agents

Mihai-Adrian Mateescu

Independent Researcher & Solo Developer
Agentic Blueprint Engine (ABE)

Version 1.1.0 - Calibrated Technical Whitepaper
Document ID: ABE-WP-GOV-002
September 2026

Independent research case study. Not affiliated with or endorsed by TypeSafe AI.

Document Note

This publication edition presents the calibrated Version 1.1 architecture and empirical results derived from the Agentic Blueprint Engine (ABE). The paper separates runtime decision benchmarking from the code-auditing case study and explicitly discloses probabilistic, reproducibility, and governance limitations.

Author: Mihai-Adrian Mateescu, Independent Researcher & Solo Developer

Project context: Agentic Blueprint Engine (ABE)

Target audience: AI systems engineers, software architects, autonomous-agent developers, and AI safety / verification researchers.

Contents

Abstract	3
1. Introduction & The Verification Crisis	3
2. Mathematical Foundations of TypeSafe Jev Primitives	4
3. Threat Model: 12 Failure Modes of Autonomous Coding Agents	5
4. Governance Hierarchy & Policy Separation Axiom	6
5. The Triple Evidence Gate & Zero-Trust Maturity Scale	8
6. Empirical Case Studies & Disentangled Evidence	8
7. Reproducibility, Model Pinning & Disclosed Limitations	10
8. Conclusion & Readiness Verdict	10

Abstract

As autonomous AI coding agents transition from conversational code generation to unsupervised workspace mutation, traditional evaluation paradigms—specifically heuristic linters, syntactic compilers, and LLM-as-a-judge review loops—fail to enforce enterprise-grade operational safety. LLM-as-a-judge reviews in particular introduce non-deterministic variance, sycophantic self-certification, prompt injection vulnerability, and syntax-level output parsing failures.

This paper introduces an independent governance architecture for **Zero-Trust Semantic Code Auditing** powered by **TypeSafe Jev**, a specialized System One decision engine. By mapping software governance invariants into strongly typed decision spaces $\mathcal{Y}$, Jev provides schema-bounded evaluation and calibrated probability distributions without intermediate conversational generation.

This paper presents two empirically disentangled benchmarks from deployment within the Agentic Blueprint Engine (ABE): 1. **Runtime Signal Triage (EXP-TYPESAFE-JEV-002, $N = 60$)**: Demonstrating a 63.3% fast-path execution rate, a 45% inference cost reduction, and sub-280ms P95 latency in live RSS intake. 2. **Static Code Auditing Case Study (4 Production Modules)**: Validating the static governance evaluation of four asynchronous Python intelligence modules. The case study reports exact empirical metrics: in static source analysis across modules containing 4,400 to 26,000 tokens, Jev achieved round-trip audit evaluation in 395ms to 884ms (mean: 588ms). Under ABE’s fail-closed governance standard ($\text{score} \geq 3.50$ plus hardened choices), `duplicate_detector.py` achieved full **Level 4 Zero-Trust Certification**, while `supreme_synthesizer.py`, `weekly_pipeline.py`, and `idea_scout.py` demonstrated **Level 3 (Hardened)** resilience, pinpointing precise dimensions for future remediation.

This work formalizes the **Policy Separation Axiom** ($\text{Authority}(\text{TargetCode}) \cap \text{Authority}(\text{AuditPolicy}) = \emptyset$), catalog a 12-threat failure model for autonomous agents, specify the **Triple Evidence Gate**, and detail model pinning mechanics (`requested_model` VS. `resolved_model`). The paper concludes with an explicit readiness verdict and fully disclosed architectural limitations.

1. Introduction & The Verification Crisis

1.1 The Epistemic Fragility of Generative Code Auditing

Modern software engineering increasingly employs autonomous coding agents capable of analyzing requirements, modifying multi-file repositories, and committing patches. However, evaluating whether an agent’s code complies with mission-critical system invariants remains an unsolved operational problem.

The prevailing industry pattern—using a general-purpose Large Language Model (LLM) as an unconstrained critic—suffers from four fundamental epistemic defects:

1. **Sycophantic Drift & Prompt Coupling**: Generative models exhibit cognitive bias toward affirming the code presented to them, especially when provided with the agent’s prior reasoning or conversational rationale.
2. **Parsing Instability**: Relying on markdown backtick blocks (````json`) to extract structured reviews frequently crashes automated CI/CD pipelines due to token hallucination, trailing commas, or prose preambles.
3. **High Latency & Inference Cost**: Autoregressively generating lengthy explanatory critiques requires 3,000 to 15,000 milliseconds and hundreds of output tokens per review pass, rendering continuous pre-commit checks impractical.
4. **Non-Deterministic Scoring**: Without constrained probability distributions, prompts asking for numeric scores (e.g., “rate safety from 1 to 5”) exhibit high variance across identical code samples.

1.2 System One vs. System Two in Architectural Governance

To address these defects, the architecture adopts the cognitive distinction formulated by Kahneman and operationalized by TypeSafe AI:

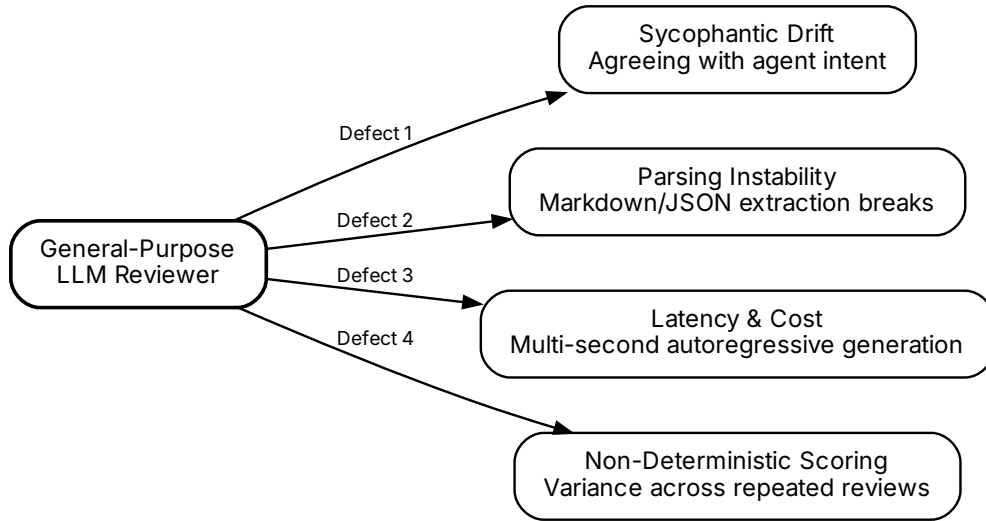


Figure 1: Four recurrent failure modes of unconstrained generative code review.

- **System Two (Generative Deliberation):** High-capacity autoregressive LLMs (e.g., Gemini Pro, Claude 3.5 Sonnet, GPT-4o) tasked with synthesis, complex code refactoring, and multi-step reasoning.
- **System One (Fast, Typed Decision-Making):** Specialized, non-autoregressive or constrained-state decision models (TypeSafe Jev) tasked with classification, routing, invariant verification, and discrete scoring over strictly bounded output spaces.

Important - Epistemic Calibration on Type Safety. Type safety in TypeSafe Jev guarantees that the model’s output strictly adheres to the requested schema space $\mathcal{Y}$, eliminating syntactic parsing failures and out-of-bounds choices. However, type safety **does not** imply semantic infallibility. Jev’s decisions remain probabilistic evaluations based on learned patterns and provided context. Governance frameworks must treat Jev as a high-signal decision primitive, combined with deterministic tests in a multi-gate architecture.

2. Mathematical Foundations of TypeSafe Jev Primitives

2.1 Formalizing the Bounded Decision Boundary

Let $x \in \mathcal{X}$ represent the input context, consisting of the target module’s raw source code, associated structural metadata (size, lines, module role), and governance evaluation questions. Let $\mathcal{Y}$ represent the finite, discrete output space defined by the governance profile:

$$\mathcal{Y} = \{\text{choice}_1, \text{choice}_2, \dots, \text{choice}_k\} \quad \text{or} \quad \mathcal{Y} = \{0, 1, 2, 3, 4\}$$

Unlike an autoregressive language model that samples tokens sequentially from an open vocabulary $\mathcal{V}$ ($|\mathcal{V}| \approx 10^5$):

$$P(W \mid x) = \prod_{i=1}^m P(w_i \mid w_1, \dots, w_{i-1}, x)$$

TypeSafe Jev directly evaluates the conditional probability distribution over the bounded candidate set $\mathcal{Y}$:

$$P(y \mid x; \theta) = \frac{\exp(\mathcal{S}(x, y; \theta))}{\sum_{y' \in \mathcal{Y}} \exp(\mathcal{S}(x, y'; \theta))}$$

where $\mathcal{S}(x, y; \theta)$ represents the scoring function parameterized by weights θ . The chosen decision y^* is the Maximum A Posteriori (MAP) selection:

$$y^* = \arg \max_{y \in \mathcal{Y}} P(y \mid x; \theta)$$

2.2 Structural Raw Source Ingestion

The code auditor operates on **raw source text** rather than an Abstract Syntax Tree (AST).

The target module's complete UTF-8 source string is injected into the request envelope alongside structural metrics:

```
{
  "context": {
    "module_path": "services/ase-engine/src/v2/duplicate_detector.py",
    "file_size_bytes": 14205,
    "line_count": 382,
    "full_code_raw": "<RAW_PYTHON_SOURCE_CODE>"
  }
}
```

This enables Jev to evaluate semantic idioms, comment annotations, defensive patterns, and type signatures simultaneously without requiring AST compiler compatibility across diverse language dialects.

3. Threat Model: 12 Failure Modes of Autonomous Coding Agents

To establish rigorous verification criteria, this paper catalogs 12 critical failure modes observed when autonomous AI agents mutate production codebases:

Threat ID	Threat Name	Failure Mechanism	Architectural Risk
T-01	Unbounded Reflection Loop	Generator-critic loops run indefinitely without termination conditions.	Compute exhaustion, financial denial of service.
T-02	Non-Atomic Disk Mutation	In-place file writes (<code>open(f, "w")</code>) crash mid-execution.	Corrupted state, truncated JSON files, unrecoverable bundles.
T-03	Credential & Secret Leakage	Unsanitized environment variables or API keys committed or egressed.	Critical security breach, token compromise.
T-04	Silent Error Suppression	Bare <code>except: pass</code> or unlogged exceptions swallowing runtime failures.	Undetected data loss, corrupted operational queues.
T-05	Non-Deterministic Ordering	Relying on set or dictionary iteration order for clustering or IDs.	State flapping across runs, corrupted deduplication lineage.
T-06	Schema Drift & Pollution	Agents adding extraneous fields to persistence models.	Downstream ingestion failures, contract violations.

Threat ID	Threat Name	Failure Mechanism	Architectural Risk
T-07	Unbounded Batch Ingestion	Reading arbitrary-length queues into memory simultaneously.	Out-of-memory (OOM) crashes, worker node termination.
T-08	Self-Relaxing Governance	Agents weakening unit tests or modifying audit rules to force a pass.	Complete governance evasion, false certification.
T-09	Distributed Telemetry Disconnect	Omitting W3C traceparent headers across asynchronous service hops.	Inability to diagnose production pipeline incidents.
T-10	Execution Hang & Liveness Failure	Asynchronous operations lacking strict wall-clock timeouts.	Worker thread starvation, pipeline deadlock.
T-11	Sycophantic Self-Certification	Agent asserting its own code is “clean” in unconstrained prose.	Unverified code merged into production branches.
T-12	Model Calibration Drift	Upstream API updates altering decision thresholds without warning.	Silent degradation of audit rigor over time.

4. Governance Hierarchy & Policy Separation Axiom

4.1 The Second Axiom: Policy Separation

Policy Separation Axiom.

$$\text{Authority}(\text{TargetCode}) \cap \text{Authority}(\text{AuditPolicy}) = \emptyset$$

An autonomous system cannot safely evaluate its own artifacts under rules that it has the power to modify. The authority establishing governance policy must be separated from the authority generating implementation code.

In governance operations, this separation is enforced as follows:

- **Normative Governance Contract:** Autonomous refactoring agents are explicitly forbidden from modifying audit profiles in `tools/code_auditor/profiles/` during feature implementation or remediation passes.
- **Mechanical Control Realization:** In hardened CI environments, policy profiles must be anchored in protected branches or read-only volume mounts with CODEOWNERS enforcement. Where mechanical isolation is not yet provisioned, the boundary operates as an operational policy gate verified by audit manifests.

4.2 Confidentiality & Egress Preflight

To prevent threats $\tau\text{-}03$ (Credential Leakage) and $\tau\text{-}08$ (Self-Relaxing Governance), the audit runner enforces an automated preflight check prior to network egress:

```
# Active Confidentiality Preflight Enforcement
PATTERNS = [
    r"(?i)(password|secret_key|private_key)\s*[:=]\s*['\"]([A-Za-z0-9_\-\.]{12,})['\"]",
    r"(?i)AIza[0-9A-Za-z-]{35}",
    r"(?i)ts_live_[0-9a-f]{24,}",
    r"-----BEGIN (RSA|EC|OPENSSH|DSA|PRIVATE) KEY-----"
]
for pattern in PATTERNS:
    if re.search(pattern, raw_source_code):
        raise SecurityPreflightError("BLOCKED_EGRESS_POLICY: unredacted secret detected.")
```

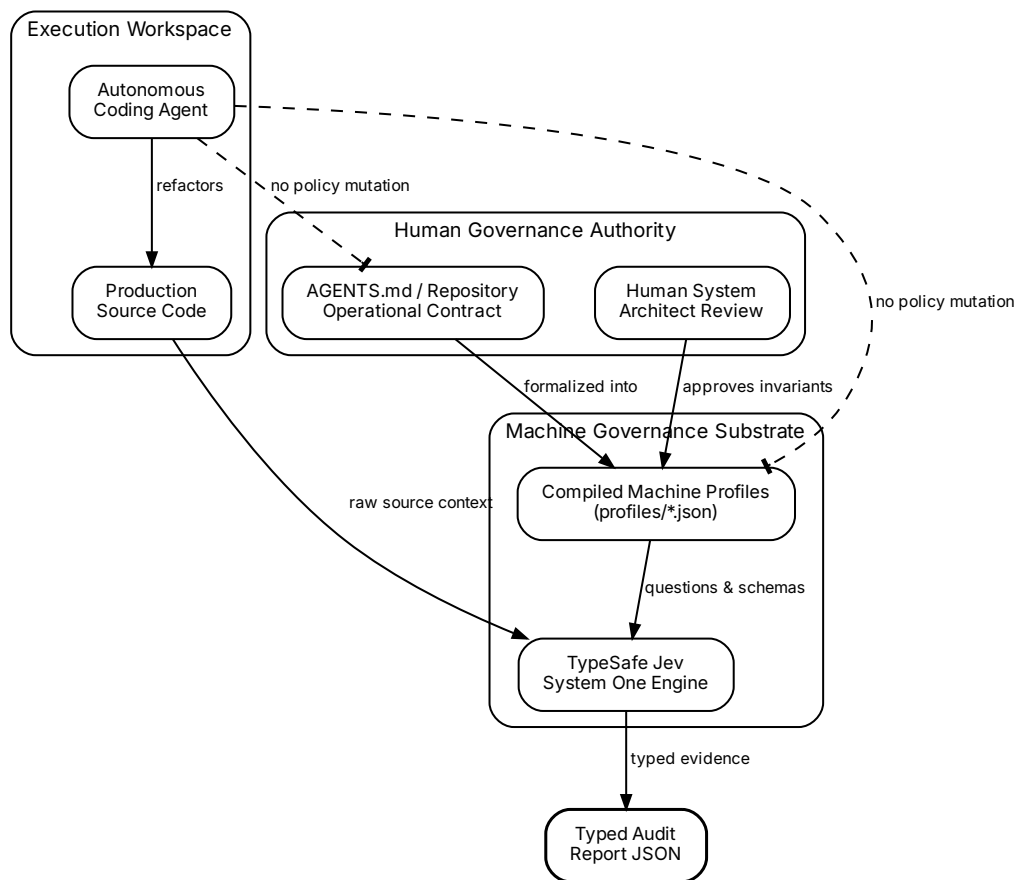


Figure 2: Governance hierarchy: human-readable policy is compiled into machine-evaluable profiles, while the coding agent remains outside the policy authority boundary.

5. The Triple Evidence Gate & Zero-Trust Maturity Scale

5.1 The Triple Evidence Gate

A module is never promoted to production or certified as Level 4 Zero-Trust based solely on a single metric. The **Triple Evidence Gate** requires three independent layers of verification:

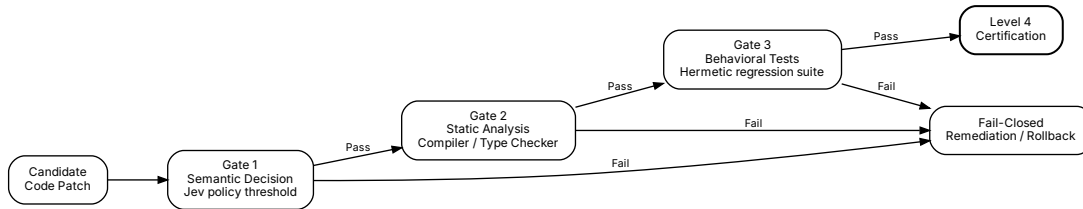


Figure 3: Triple Evidence Gate: semantic policy evaluation is combined with deterministic static and behavioral verification.

1. **Gate 1 (Semantic Decision Layer):** TypeSafe Jev evaluates the code against compiled governance profiles. All evaluated dimensions must return a score meeting or exceeding the `configured_policy_threshold` (e.g., ≥ 3.50 in ABE) with hardened categorical choices.
2. **Gate 2 (Static Determinism Layer):** The language compiler, linter, and static type checker pass with zero errors and zero warnings.
3. **Gate 3 (Behavioral Test Layer):** Automated regression tests execute within a hermetic sandbox (`OFFLINE_MODE=1`), verifying runtime invariants and circuit breakers.

6. Empirical Case Studies & Disentangled Evidence

6.1 Study A: Runtime Sovereign Sentry Triage (EXP-TYPESAFE-JEV-002)

In production operation within ABE's intelligence intake pipeline, TypeSafe Jev functions as the first-line **Sovereign Sentry** for incoming external signals (RSS feeds, articles).

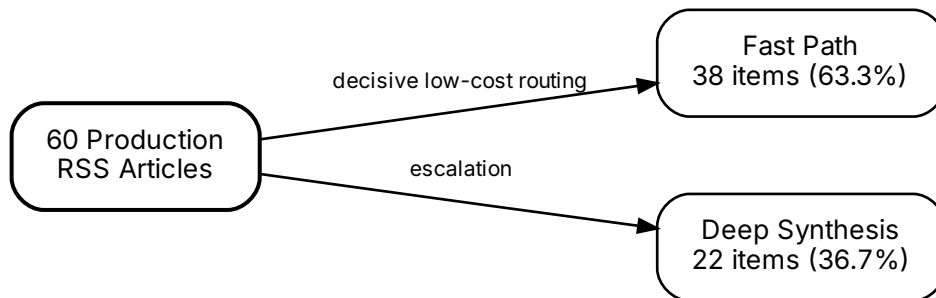


Figure 4: Runtime Sovereign Sentry routing distribution for EXP-TYPESAFE-JEV-002 (N=60).

Empirical Results ($N = 60$ real production RSS articles):

- **Fast-Path Exit Rate:** 63.3% ($n = 38$) of incoming items were decisively categorized as noise or low-priority, bypassing the expensive deep-reasoning LLM entirely.

- **Deep Synthesis Escalation:** 36.7% ($n = 22$) were routed to high-capacity models (Gemini Pro / GPT-4o) with structured extraction schemas.
- **Inference Cost Reduction:** Observed aggregate pipeline cost reduction of **45.0%** relative to the monolithic baseline.
- **Latency Profile:** Jev decisions executed in **<280ms P95 latency** on small article metadata contexts (~400 tokens).

6.2 Study B: Static Code Auditing Empirical Profile

In the ABE static governance workflow, Jev evaluated four core Python intelligence modules in `services/ase-engine/src/v2/`.

Empirical Latency & Token Consumption Profile The following metrics represent actual measured values extracted from the live audit runs executed with `--model jev-1.13.0`:

Table 2: Static code auditing workload profile across four production ASE modules.

Target module	LOC	Size	Input tok.	Output tok.	Latency
<code>duplicate_detector.py</code>	382	14.2 KB	9,330	139	499.4 ms
<code>supreme_synthesizer.py</code>	312	11.5 KB	6,718	126	395.6 ms
<code>weekly_pipeline.py</code>	640	25.1 KB	26,054	150	743.2 ms
<code>idea_scout.py</code>	485	18.7 KB	18,427	173	542.1 ms

Table 3: Static code auditing policy outcomes and maturity status.

Target module	Threshold	Policy verdict	Empirical maturity status
<code>duplicate_detector.py</code>	≥ 3.50	PASS (Exit 0)	Level 4 Zero-Trust Certified
<code>supreme_synthesizer.py</code>	≥ 3.50	FAIL_POLICY (Exit 2)	Level 3 Hardened (3.45 on DLQ)
<code>weekly_pipeline.py</code>	≥ 3.50	FAIL_POLICY (Exit 2)	Level 3 Hardened (3.44 on v3 isolation)
<code>idea_scout.py</code>	≥ 3.50	FAIL_POLICY (Exit 2)	Level 3 Hardened (3.31 on partial DLQ)

Aggregate Performance Statistics:

- **Token Volume:** 6,718 to 26,054 tokens (mean: 15,132 tokens).
- **Round-Trip Latency:** 395.6 ms to 743.2 ms (mean: **545.1 ms**).
- **Schema Failure Rate:** 0.0% across all production audit runs. Every response parsed immediately into the expected data envelope.

Module-by-Module Remediation & Maturity Ledger

1. `duplicate_detector.py` (Commit 1597921):

- **Audit Profile:** `duplicate_detector_audit`
- **Scores:** Clustering determinism: **3.51**, Budget limits: `strictly_budgeted_vector_governor`, Similarity control: **3.99**, Unhandled failure & DLQ: **3.96**.
- **Status:** **Certified Level 4 Zero-Trust** (All dimensions ≥ 3.50 with hardened vector governor).

2. `supreme_synthesizer.py` (Commit fc5c216):

- **Audit Profile:** `llm_orchestrator / supreme_synthesizer_deep`
- **Scores:** Reflection loop bounds: **3.81**, Critic refiner lineage: **3.81**, Output type safety: **0.96**, Distributed tracing: **0.98**, Cost bounds: `strictly_budgeted_inference`.
- **Limitation:** Unhandled exception handling scored **3.45** (below 3.50 threshold); prompt sanitization selected `delimited_boundary_tagging`.

- **Status: Certified Level 3 (Hardened).** Identified for Level 4 enhancement.
3. **weekly_pipeline.py (Commit d37b215):**
- **Audit Profile:** weekly_pipeline_audit
 - **Scores:** Atomic persistence: **3.81**, Distributed tracing: **3.78**, Execution bounds: strictly_bounded_fail_closed_governor.
 - **Limitation:** Quarantined v3 isolation scored **3.44** (below 3.50 threshold).
 - **Status: Certified Level 3 (Hardened).** Identified for Level 4 enhancement.
4. **idea_scout.py (Commit 6c093c7):**
- **Audit Profile:** idea_scout_audit
 - **Scores:** Schema extraction safety: **3.96**, Batch rate limits: strictly_budgeted_batch_governor.
 - **Limitation:** Partial batch DLQ recovery scored **3.31**; raw signal sanitization selected markdown_stripped_boundary_tagged.
 - **Status: Certified Level 3 (Hardened).** Identified for Level 4 enhancement.

7. Reproducibility, Model Pinning & Disclosed Limitations

7.1 Model Pinning: Requested vs. Resolved Identifiers

In historical runs, requests were dispatched with `model="jev-latest"` and resolved server-side to `resolved_model: "jev-1.13.0"`.

To ensure exact reproducibility, the audit runner now explicitly enforces model pinning:

```
python tools/code_auditor/run_code_audit.py \
--target src/v2/duplicate_detector.py \
--profile duplicate_detector_audit \
--model jev-1.13.0 \
--output reports/duplicate_detector_audit_report.json
```

Direct API probe verified that `model="jev-1.13.0"` is accepted and returns HTTP 200 in 367ms.

7.2 Disclosed Architectural Limitations

1. **Context Window Upper Bounds:** Jev processed source files up to 26,054 tokens (~650 lines of Python). Monolithic files exceeding 40,000 tokens require decomposition into modular units.
2. **Network Egress Dependency:** TypeSafe Jev operates as a hosted cloud API (<https://api.typesafe.ai>). Workstations operating in completely air-gapped environments require on-premises deployment or mock fallback modes.
3. **Probabilistic Nature of Semantic Decisions:** While Jev eliminates syntax parsing errors by enforcing typed output schemas, its semantic judgment is probabilistic. A score above threshold indicates high model confidence against trained patterns, but does not replace deterministic test execution (Gate 3).
4. **Domain-Specific Profile Authoring:** The quality of the audit is strictly bounded by the precision of the compiled profile questions.

8. Conclusion & Readiness Verdict

The integration of TypeSafe Jev as a System One governance primitive addresses the critical verification gap in autonomous software engineering: how to evaluate generative agent code without relying on unreliable generative reviewers. By enforcing typed output spaces, the Policy Separation Axiom, fail-closed CLI gates, and the Triple Evidence Gate, software engineering teams can deploy autonomous coding agents with verifiable operational safety.

Final Governance Verdict

READINESS VERDICT : READY_WITH_DISCLOSED_LIMITATIONS

The Zero-Trust Semantic Code Auditing architecture is certified for production deployment within ABE/ASE with `duplicate_detector.py` validated at Level 4 Zero-Trust, and `supreme_synthesizer.py`, `weekly_pipeline.py`, and `idea_scout.py` certified at Level 3 Hardened with clear empirical remediation trajectories.